

Documentación Técnica Completa

Sistema Distribuido de Restaurantes

Cesar Gamboa

Daniel Morera

Junio 2026

Resumen

El sistema *Restaurante-DB-I-Proyecto* es una aplicación distribuida orientada a resolver la gestión integral de restaurantes, usuarios, menús, productos, reservaciones y pedidos dentro de un entorno digital. La solución permite centralizar procesos que normalmente se realizan de forma separada, como el registro de clientes y administradores, la administración de restaurantes, la consulta de menús, la disponibilidad de mesas, la creación de reservaciones, la realización de pedidos y la búsqueda eficiente de productos.

La aplicación busca mejorar la organización operativa de un restaurante al ofrecer una plataforma capaz de manejar autenticación segura, control de roles, persistencia de datos, búsqueda especializada, cacheo de información frecuente y escalabilidad de servicios. Para esto, el sistema integra tecnologías como FastAPI, Keycloak, PostgreSQL, MongoDB, Redis, Elasticsearch, Nginx, Docker Compose y Kubernetes, permitiendo que los servicios trabajen de forma separada pero coordinada dentro de una arquitectura contenerizada.

Además de resolver necesidades transaccionales, el proyecto incorpora una capa analítica mediante la carpeta `dataW`, donde se implementan procesos de extracción, transformación, carga, validación y visualización de datos. Esta plataforma utiliza Apache Hive, Apache Spark, Apache Airflow y Apache Superset para construir un Data Warehouse, generar vistas OLAP y facilitar el análisis de información relacionada con ventas, pedidos, productos, clientes y comportamiento operativo del restaurante.

En conjunto, esta aplicación no solo permite administrar las operaciones principales de un restaurante, sino que también brinda herramientas para analizar los datos generados por el sistema y apoyar la toma de decisiones. Este documento consolida la explicación funcional, técnica y operativa de la solución, incluyendo su arquitectura, servicios, modelos de datos, contratos de API, flujos de información, ejecución local, pruebas y mecanismos de validación.

Índice

1. Resumen del Programa	7
2. Cómo leer esta documentación	7
2.1. Mapa de navegación	7
2.2. Convención de bloques de API	8
3. Objetivos del sistema	8
3.1. Objetivo general	8
3.2. Objetivos específicos	8
4. Fuentes documentales y artefactos base	9
5. Getting started	9
5.1. Requisitos previos	9
5.2. 1. Levantar servicios	9
5.3. 2. Revisar estado	9
5.4. 3. Abrir documentación interactiva	10
5.5. 4. Obtener token	10
5.6. 5. Hacer primera llamada autenticada	10
6. Conceptos principales	10
6.1. Access token	10
6.2. Roles	11
6.3. Base URL	11
6.4. Motor activo de base de datos	11
6.5. Cache hit y cache miss	11
6.6. Reindexado	11
6.7. Data Warehouse	11
7. Estructura general del repositorio	12
8. Arquitectura lógica	12
8.1. Capas	13
9. Arquitectura física	13
9.1. Componentes físicos	14
9.2. Volúmenes persistentes	15
10. Tecnologías utilizadas	15
10.1. FastAPI	15
10.2. Keycloak	15
10.3. PostgreSQL	16
10.4. MongoDB shardeado	16
10.5. Redis	16

10.6. Elasticsearch	16
10.7. Nginx	17
10.8. Docker Compose	17
10.9. Kubernetes	17
10.10. Apache Hive	17
10.11. Apache Spark	17
10.12. Apache Superset	17
10.13. Apache Airflow	18
10.14. Neo4J	18
11. Configuración principal	18
12. Modelo de datos transaccional	18
12.1. Entidades	18
12.2. Restricciones relevantes	19
13. Capa DAO	19
14. Autenticación y autorización	19
14.1. Flujo de registro	20
14.2. Flujo de login	20
14.3. Validación de token	20
14.4. Roles	20
15. Convenciones generales de API	20
15.1. Base URLs	20
15.2. Headers	21
15.3. Códigos de estado	21
16. How-Tos	21
16.1. Registrar un cliente	21
16.2. Crear un plato de menú	21
16.3. Buscar productos	22
16.4. Reindexar Elasticsearch	22
16.5. Ejecutar el pipeline analítico	22
17. API Reference	22
17.1. Auth	22
17.1.1. POST /auth/register	22
17.1.2. POST /auth/login	23
17.2. Users	23
17.2.1. GET /users/me	23
17.2.2. PUT /users/{user_id}	24
17.2.3. DELETE /users/{user_id}	24
17.3. Restaurants	24
17.3.1. GET /restaurants/	24

17.3.2. GET /restaurants/{restaurant_id}	24
17.3.3. POST /restaurants/	25
17.3.4. PUT /restaurants/{restaurant_id}	25
17.3.5. DELETE /restaurants/{restaurant_id}	25
17.4. Menus	25
17.4.1. GET /menus/	25
17.4.2. GET /menus/{menu_id}	25
17.4.3. POST /menus/	26
17.4.4. PUT /menus/{menu_id}	26
17.4.5. DELETE /menus/{menu_id}	26
17.5. Tables	27
17.5.1. GET /tables/	27
17.5.2. POST /tables/	27
17.5.3. GET /tables/{table_id}	27
17.5.4. PUT /tables/{table_id}	27
17.5.5. DELETE /tables/{table_id}	27
17.6. Orders	27
17.6.1. POST /orders/	27
17.6.2. GET /orders/{order_id}	28
17.7. Reservations	28
17.7.1. POST /reservations/	28
17.7.2. DELETE /reservations/{reservation_id}	28
17.8. Search	28
17.8.1. GET /products	28
17.8.2. GET /products/category/{category}	29
17.8.3. POST /reindex	29
17.9. Graph	29
17.9.1. GET /graph/export	29
18. Contratos de API	30
18.1. Sistema	30
18.2. Autenticación	30
18.2.1. POST /auth/register	30
18.2.2. POST /auth/login	30
18.3. Usuarios	31
18.4. Restaurantes	32
18.5. Menús	32
18.6. Mesas	33
18.7. Pedidos	33
18.8. Reservaciones	34
18.9. Búsqueda	34
18.10. Exportación para grafo	35
18.10.1. GET /graph/export	35

19.Flujos de datos operacionales	36
19.1. Administrador agrega plato	36
19.2. Cliente busca productos	37
19.3. Cliente crea reservación	38
20.Ejecución local	38
20.1. Levantar el sistema completo	38
20.2. Verificar servicios	38
20.3. Credenciales por defecto	38
21.Carpeta dataW: plataforma analítica completa	39
21.1. Objetivo de dataW	39
21.2. Estructura de dataW	40
21.3. Data Warehouse en Hive	41
21.3.1. Dimensiones	41
21.3.2. Hechos	41
21.3.3. Vistas OLAP actuales	41
21.4. Generación de seed operacional	43
21.5. Spark	43
21.5.1. Entradas simuladas y escalamiento	43
21.5.2. Salidas	43
21.6. Superset	43
21.6.1. Dashboards requeridos	44
21.7. Airflow	44
21.7.1. Tareas del DAG	44
21.7.2. Descripción de tareas	44
22.Neo4J y asignación de rutas	45
22.1. Modelo de grafo	45
22.2. Heurística de vecino más cercano	45
23.Pruebas y validación	45
23.1. Pruebas unitarias	45
23.2. Pruebas de integración	45
23.3. Validación Spark	46
23.4. Validación Data Warehouse	46
23.5. Validación visualización	46
23.6. Validación Airflow	46
23.7. Validación Neo4J	46
24.Seguridad	46
25.Consideraciones de consistencia	46
26.Limitaciones conocidas	47
27.Conclusión	47

28.Apéndice A: resumen de endpoints	47
29.Apéndice B: archivos fuente relevantes	48
30.Referencias tecnológicas	48
31.Glosario de acrónimos	49
32.Referencias tecnológicas	51

1 Resumen del Programa

El proyecto *Restaurante-DB-I-Proyecto* implementa una plataforma distribuida para la gestión integral de restaurantes. Desde una perspectiva de negocio, la aplicación busca resolver la falta de centralización en procesos como la administración de usuarios, restaurantes, menús, productos, mesas, reservaciones, pedidos y búsqueda de información. En muchos restaurantes, estas tareas pueden manejarse de forma separada o manual, lo que dificulta la organización, la trazabilidad de las operaciones, la atención al cliente y el análisis posterior de los datos generados por el negocio.

La solución permite que los clientes consulten restaurantes, revisen menús, realicen reservaciones y generen pedidos, mientras que los administradores pueden gestionar restaurantes, productos, mesas y operaciones asociadas. Con esto, la aplicación mejora la comunicación entre clientes y restaurantes, ordena los procesos internos y proporciona una base de datos estructurada para consultar, proteger y analizar la información del negocio.

El sistema se divide en dos grandes dominios. El **dominio transaccional** cubre la operación diaria mediante una API REST en FastAPI, autenticación con Keycloak, persistencia seleccionable entre PostgreSQL y MongoDB shardeado, caché con Redis, búsqueda con Elasticsearch y entrada unificada con Nginx. Este dominio permite administrar las funcionalidades principales del sistema, como usuarios, restaurantes, menús, reservaciones y pedidos.

El **dominio analítico** se encuentra en la carpeta `dataW` e incorpora un Data Warehouse en Hive, procesamiento con Spark, dashboards en Superset y orquestación con Airflow. Su objetivo es transformar los datos operacionales en información útil para analizar ventas, productos, clientes, pedidos y tendencias de consumo.

El sistema corre principalmente sobre Docker Compose y también incluye una definición de despliegue en Kubernetes para los servicios principales. Además, la API puede trabajar contra PostgreSQL o MongoDB según la variable `DATABASE_ENGINE`, manteniendo una capa DAO común para desacoplar la lógica de negocio del motor de base de datos utilizado.

2 Cómo leer esta documentación

Esta documentación está organizada con un estilo similar a un portal de documentación para desarrolladores, se tomo como referencia la web oficial de Spotify. Primero se presenta una visión general, luego una guía de inicio, conceptos clave, guías prácticas y finalmente una referencia de API detallada. En la documentación oficial de Spotify, la Web API separa *Overview*, *Getting started*, *Concepts*, *Tutorials*, *How-Tos* y *Reference*; cada endpoint de referencia muestra método, ruta, autenticación, parámetros, respuesta y códigos de error.

2.1 Mapa de navegación

Sección	Qué resuelve
Overview	Explica qué es el sistema, qué problema resuelve y cuáles son sus componentes.

Getting started	Indica cómo levantar el proyecto, obtener token y hacer la primera llamada.
Concepts	Define tokens, roles, motores de base de datos, caché, búsqueda, sharding y pipeline analítico.
How-Tos	Muestra tareas comunes: registrar usuario, crear restaurante, reindexar búsqueda, correr Airflow y validar <code>dataW</code> .
Reference	Documenta cada endpoint con request, parámetros, body, respuesta, errores e interpretación.

2.2 Convención de bloques de API

Cada endpoint se documenta con la siguiente plantilla:

- **Endpoint:** método HTTP y ruta.
- **Descripción:** propósito funcional.
- **Autenticación:** si requiere token y qué rol necesita.
- **Request:** path params, query params, headers y body.
- **Response:** códigos esperados y estructura de salida.
- **Errores:** significado de cada caso de fallo.
- **Interacción interna:** servicios que toca: Keycloak, DAO, Redis, Elasticsearch, PostgreSQL, MongoDB, Hive o Neo4J.

3 Objetivos del sistema

Antes de definir los objetivos específicos del sistema, es importante contextualizar el propósito general de la aplicación. Estos objetivos permiten establecer con claridad qué necesidades busca cubrir el proyecto, tanto desde el punto de vista funcional como desde su aporte a la gestión operativa y analítica de restaurantes.

3.1 Objetivo general

Construir una plataforma distribuida de gestión de restaurantes que permita administrar entidades operacionales, autenticar usuarios por roles, ofrecer búsqueda textual eficiente, cachear datos consultados con frecuencia y generar análisis OLAP mediante un flujo de datos completo.

3.2 Objetivos específicos

1. Exponer una API REST segura para autenticación, usuarios, restaurantes, menús, mesas, pedidos y reservaciones.
2. Permitir el uso intercambiable de PostgreSQL o MongoDB como motor activo de persistencia.
3. Implementar MongoDB en modo shardeado con config servers, shard replica set y router mongos.
4. Integrar Keycloak para registro, login, emisión y validación de tokens JWT.

5. Utilizar Redis para reducir accesos repetidos a la base de datos y a Elasticsearch.
6. Utilizar Elasticsearch para búsqueda textual sobre productos del menú.
7. Publicar servicios detrás de Nginx mediante rutas `/api` y `/search`.
8. Construir un Data Warehouse OLAP con esquema estrella en Hive.
9. Procesar datos con Spark usando DataFrames y SparkSQL.
10. Visualizar indicadores en Superset.
11. Orquestrar el pipeline analítico con Airflow.
12. Modelar y validar asignación de rutas con Neo4J y una heurística de vecino más cercano.

4 Fuentes documentales y artefactos base

Esta documentación consolida información técnica del repositorio y de los artefactos ubicados en `documentation/`. Los documentos y diagramas base son:

- `documentation/_BD2_PY01__Restaurantes.pdf`: enunciado/guía de la primera etapa del proyecto.
- `documentation/_BD2_PY02__Restaurantes_2.pdf`: enunciado/guía de la segunda etapa del proyecto.
- `documentation/_BD2_TC01b__Docker_y_RestAPI.pdf`: material de referencia sobre Docker y REST API.
- `documentation/Restaurantese2-CesarGamboa,DanielMorera.pdf`: documento previo de entrega.

5 Getting started

Esta sección permite levantar el sistema y realizar una primera llamada autenticada. La idea es que cualquier persona nueva en el proyecto pueda pasar de cero a una petición funcional sin leer todavía toda la arquitectura.

5.1 Requisitos previos

- Docker Desktop con Docker Compose.
- PowerShell en Windows.
- Puertos disponibles: 5432, 6379, 8000, 8001, 8080, 9200, 27017, 8088, 8090, 10000.
- Archivo `.env` en la raíz del proyecto.

5.2 1. Levantar servicios

```
docker compose up -d --build
```

5.3 2. Revisar estado

```
docker compose ps
docker compose logs -f keycloak
docker compose logs -f api
```

5.4 3. Abrir documentación interactiva

- API directa: <http://localhost:8000/docs>
- API por Nginx: <http://localhost:8080/api/docs>
- Search por Nginx: <http://localhost:8080/search/docs>

5.5 4. Obtener token

```
curl -X POST http://localhost:8000/auth/login ^
-H "Content-Type: application/json" ^
-d "{\"username\":\"admin\",\"password\":\"admin\"}"
```

La respuesta incluye `access_token`. Ese token se usa como:

```
Authorization: Bearer <access_token>
```

5.6 5. Hacer primera llamada autenticada

```
curl http://localhost:8000/users/me ^
-H "Authorization: Bearer <access_token>"
```

Si el token es válido y el usuario existe en la base activa, la API responde con los datos locales del usuario.

6 Conceptos principales

Para facilitar la lectura de la documentación, esta sección introduce los términos técnicos más importantes utilizados en el proyecto. Estos conceptos ayudan a comprender cómo se manejan la autenticación, los roles de usuario, la comunicación con la API, el uso de caché, el reindexado de búsquedas y el componente analítico del sistema.

6.1 Access token

El `access_token` es un JWT emitido por Keycloak. La API no acepta usuario y contraseña en cada endpoint; acepta un token firmado. El token identifica al usuario mediante el claim `sub` y contiene roles para autorización.

6.2 Roles

- **client:** usuario cliente. Puede consultar datos protegidos, buscar productos, crear pedidos y crear reservaciones.
- **admin:** usuario administrador. Puede crear, actualizar y eliminar restaurantes, menús y mesas.

6.3 Base URL

La misma API puede consumirse por dos caminos:

- **Directo:** `http://localhost:8000`
- **Proxy Nginx:** `http://localhost:8080/api`

El servicio de búsqueda se publica por Nginx como:

- `http://localhost:8080/search`

6.4 Motor activo de base de datos

La API usa un motor activo a la vez. La variable `DATABASE_ENGINE` define si las operaciones se ejecutan sobre PostgreSQL o MongoDB. Los routers no cambian; lo que cambia es la implementación del DAO.

6.5 Cache hit y cache miss

Cuando se consulta una lista de menús o una búsqueda, Redis puede devolver una respuesta guardada:

- **Cache hit:** Redis tenía el resultado y se evita consultar base o Elasticsearch.
- **Cache miss:** Redis no tenía el resultado; la API consulta la fuente real y luego guarda la respuesta.

6.6 Reindexado

El endpoint `POST/reindex` reconstruye el índice de Elasticsearch desde los menús de la base activa. Debe ejecutarse cuando hay cambios de catálogo que no fueron indexados automáticamente o cuando Airflow detecta cambios en `dim_product`.

6.7 Data Warehouse

El Data Warehouse no reemplaza la base transaccional. Es una copia analítica en Hive optimizada para agregaciones, cubos y dashboards.

7 Estructura general del repositorio

La siguiente tabla resume la distribución principal del repositorio y explica la función de las carpetas y archivos más importantes. Esta estructura permite ubicar fácilmente los componentes operativos, analíticos, de despliegue y documentación del sistema.

Ruta	Propósito
backend/	Código Python de la API FastAPI, modelos SQLAlchemy, esquemas Pydantic, routers, DAO, autenticación, caché y búsqueda.
db/postgres/	Scripts de inicialización de PostgreSQL.
db/mongo/	Scripts de inicialización del cluster shardeado MongoDB, colecciones, índices y shard keys.
nginx/	Configuración de proxy reverso Nginx.
dataW/	Plataforma analítica: Data Warehouse, Spark, Superset, Airflow, pruebas y evidencias.
neo4j/	Carga de grafo, consultas Cypher y asignación de rutas de entrega.
kubernetes/	Manifiestos YAML y scripts PowerShell para despliegue en Kubernetes local.
documentation/	Diagramas, enunciados PDF y esta documentación LaTeX.
docker-compose.yml	Orquestación local de servicios transaccionales, analíticos y de grafo.
README.md	Guía rápida de ejecución, URLs, credenciales y pruebas.

8 Arquitectura lógica

La arquitectura lógica representa las capas del sistema y sus responsabilidades.

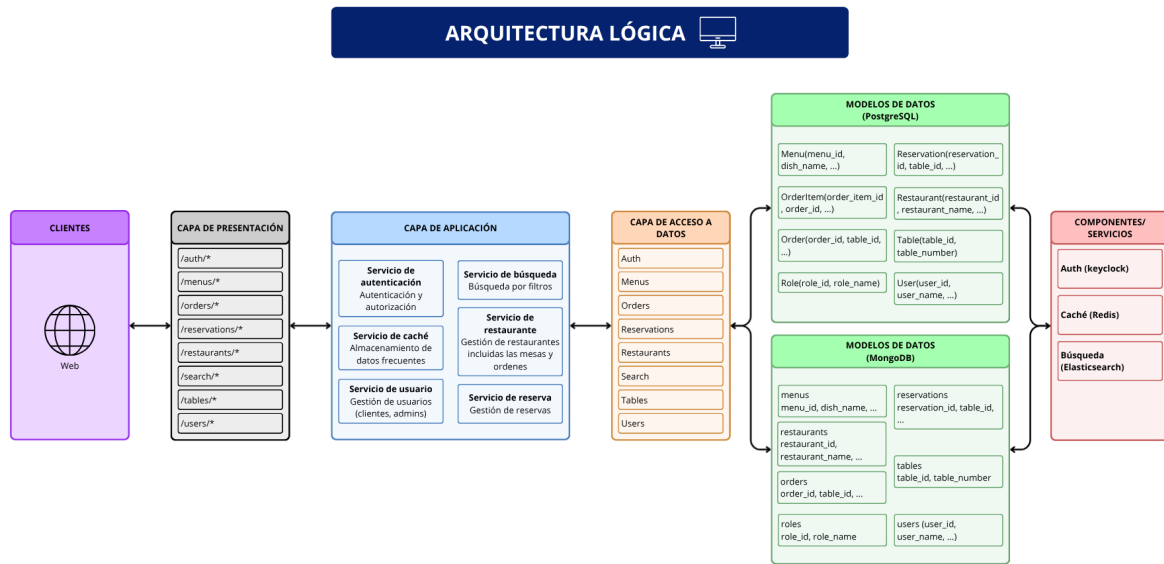


Figura 1: Arquitectura lógica del sistema.

8.1 Capas

1. **Clientes:** aplicaciones web o clientes HTTP que consumen endpoints REST.
2. **Capa de presentación:** endpoints HTTP expuestos por FastAPI y publicados por Nginx.
3. **Capa de aplicación:** routers, validaciones de negocio, autenticación, autorización, caché y búsqueda.
4. **Capa de acceso a datos:** DAO común con implementaciones para PostgreSQL y MongoDB.
5. **Modelos de datos:** entidades transaccionales como usuarios, roles, restaurantes, menús, mesas, pedidos, ítems y reservaciones.
6. **Servicios externos:** Keycloak, Redis y Elasticsearch.
7. **Capa analítica:** Hive, Spark, Superset y Airflow dentro de dataW.

9 Arquitectura física

La arquitectura física define contenedores, red interna, puertos, servicios externos expuestos y volúmenes persistentes. Todos los contenedores principales se conectan a la red Docker `restaurant-network`.

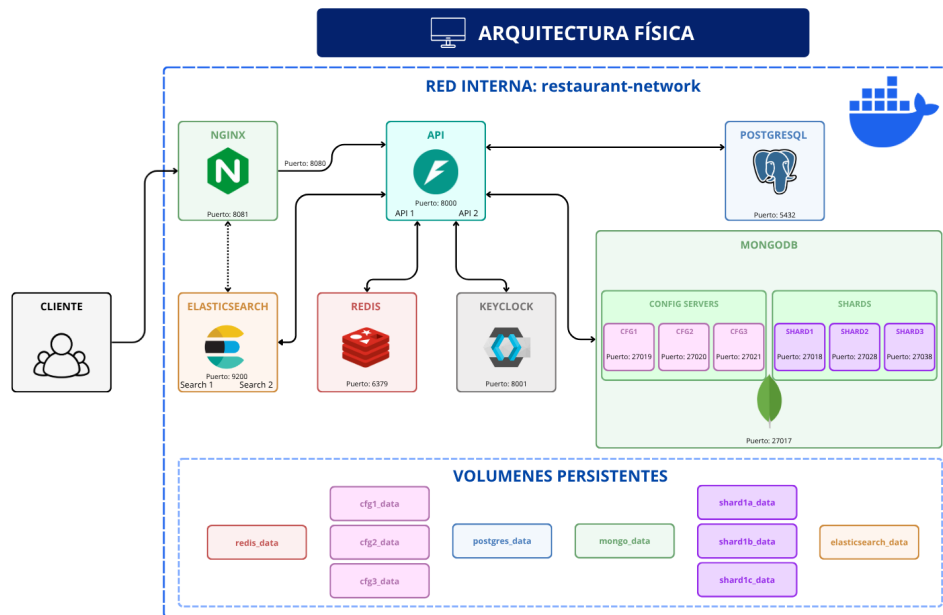


Figura 2: Arquitectura física del sistema.

9.1 Componentes físicos

Los puertos de la siguiente tabla corresponden a los valores por defecto de `docker-compose.yml`. Si se sobrescriben variables en `.env`, los puertos locales pueden cambiar. Cuando se indica *interno*, significa que el servicio escucha dentro de la red Docker y normalmente se consume a través de Nginx u otro contenedor, no directamente desde el host.

Servicio	Puerto local	Función
Nginx	8080	Entrada HTTP unificada. Redirige <code>/api/</code> hacia API y <code>/search/</code> hacia Search.
API FastAPI	8000 directo, interno 80	Servicio transaccional principal.
Search FastAPI	interno 80	Microservicio especializado para búsqueda y reindexado.
Keycloak	8001	Identidad, roles, emisión de JWT y consola administrativa.
PostgreSQL	5432	Base relacional alternativa para datos transaccionales.
MongoDB mongos	27017	Router del cluster shardeado MongoDB.
Redis	6379	Caché de menús y búsquedas.
Elasticsearch	9200	Índice textual de productos.
Spark Master	7077, UI 8081	Coordinador de procesamiento Spark.
Spark Worker	UI 8082	Ejecutor de tareas Spark.

HiveServer2	10000, UI 10002	Data Warehouse OLAP.
Superset	8088	Visualización BI y dashboards.
Airflow	8090	Orquestación del pipeline analítico.
Neo4J	7474, 7687	Grafo y rutas de entrega.

9.2 Volúmenes persistentes

El sistema utiliza volúmenes Docker para conservar datos aunque se reinicien contenedores. Los más relevantes son:

- `postgres_data`: datos de PostgreSQL.
- `cfg1_data`, `cfg2_data`, `cfg3_data`: config servers de MongoDB.
- `shard1a_data`, `shard1b_data`, `shard1c_data`: shard replica set de MongoDB.
- `redis_data`: persistencia de Redis.
- `elasticsearch_data`: índice de Elasticsearch.
- `hive_data` y `hive_metastore`: datos y metastore de Hive.
- `superset_home`: metadata y configuración de Superset.
- `airflow_postgres_data`: metadata de Airflow.
- `neo4j_data`: base de datos de Neo4J.

10 Tecnologías utilizadas

El sistema está compuesto por un conjunto de tecnologías que permiten cubrir necesidades funcionales, operativas y analíticas. Estas herramientas fueron seleccionadas para facilitar la separación de responsabilidades, la escalabilidad, la seguridad, la persistencia de datos, el procesamiento analítico y la administración de servicios contenerizados. A continuación, se describe el papel de cada tecnología dentro del proyecto.

10.1 FastAPI

FastAPI¹ expone los endpoints REST. Usa Pydantic para validar entradas y salidas, dependencias para inyectar autenticación y DAO, y documentación automática OpenAPI en `/docs`. El proyecto usa dos aplicaciones principales:

- `backend/app/api_main.py`: API transaccional.
- `backend/app/search_main.py`: microservicio de búsqueda.

10.2 Keycloak

Keycloak² administra usuarios, roles y tokens JWT. El login se realiza contra el endpoint OpenID Connect de Keycloak usando `grant_type=password`. Las rutas protegidas reciben el token en:

```
Authorization: Bearer <access_token>
```

La API valida el JWT consultando las llaves públicas JWKS del realm configurado. Si el token es válido, el payload se usa para identificar al usuario y revisar roles. Los roles esperados son `admin` y `client`.

10.3 PostgreSQL

PostgreSQL³ funciona como motor relacional. Los modelos SQLAlchemy definen tablas, claves foráneas, relaciones y restricciones únicas. Se usa cuando:

```
DATABASE_ENGINE=postgres
```

10.4 MongoDB shardeado

MongoDB⁴ funciona como motor documental distribuido. El cluster se compone de:

- Tres config servers: `cfg1`, `cfg2`, `cfg3`.
- Un shard replica set: `shard1a`, `shard1b`, `shard1c`.
- Un router: `mongos`.

La API no se conecta a los shards directamente, sino a:

```
mongodb://mongos:27017/restaurant_mongo_db
```

Las colecciones shardeadas principales son:

- `menus`: shard key `\{restaurant_id:1,dish_name:1\}`.
- `reservations`: shard key `\{reservation_id:"hashed"\}`.

10.5 Redis

Redis⁵ guarda respuestas frecuentes para reducir latencia:

- Lista completa de menús: `menus:all`.
- Menú individual: `menus:<id>`.
- Búsquedas textuales: `search:products:text:<query>`.
- Búsquedas por categoría: `search:products:category:<category>`.

El TTL general se controla con `CACHE_TTL_SECONDS`; el TTL de búsqueda se controla con `SEARCH_CACHE_TTL_SECONDS`.

10.6 Elasticsearch

Elasticsearch⁶ indexa productos del menú en el índice `products`. Cada documento contiene:

- `menu_id`
- `dish_name`
- `category`
- `description`
- `price`
- `restaurant_id`

Las búsquedas usan `multi_match` sobre nombre, categoría y descripción. El filtro por categoría usa un campo keyword normalizado en minúsculas: `category.raw`.

10.7 Nginx

Nginx⁷ opera como proxy reverso. Publica:

- `http://localhost:8080/api/` hacia el servicio `api`.
- `http://localhost:8080/search/` hacia el servicio `search`.

10.8 Docker Compose

Docker Compose⁹ levanta el ambiente completo. Además de los servicios transaccionales, integra Spark, Hive, Superset, Airflow y Neo4J. El comando base es:

```
docker compose up -d --build
```

“`latex`

10.9 Kubernetes

Kubernetes¹⁰ es una plataforma de orquestación de contenedores que permite desplegar, administrar y escalar aplicaciones distribuidas de forma automatizada. En el proyecto se utiliza como alternativa de despliegue a Docker Compose, con el objetivo de representar cómo los servicios principales podrían ejecutarse en un entorno más cercano a producción.

La carpeta `kubernetes/` contiene los manifiestos necesarios para definir los recursos del sistema, incluyendo namespace, configmaps, servicios, deployments, almacenamiento e ingress. Estos archivos permiten describir la infraestructura requerida para levantar componentes como la API, bases de datos, servicios auxiliares y puntos de entrada HTTP.

10.10 Apache Hive

Hive¹¹ implementa el Data Warehouse. El esquema estrella vive en `dataW/warehouse/schemas/schema_star.sql`. HiveServer2 permite que Superset y validaciones consulten la base `restaurant_warehouse`.

10.11 Apache Spark

Spark¹² procesa datos a escala. El job `dataW/spark/jobs/analytics_job.py` crea DataFrames, aplica transformaciones, ejecuta SparkSQL y genera salidas CSV/JSON.

10.12 Apache Superset

Superset¹³ se conecta directamente a HiveServer2 mediante la conexión:

```
hive://hive@hiveserver2:10000/restaurant_warehouse?auth=NOSASL
```

Su función es construir dashboards sobre vistas OLAP.

10.13 Apache Airflow

Airflow¹⁴ orquesta el pipeline analítico diario `restaurant_olap_pipeline`. Coordina extracción, Spark, carga a Hive, validación OLAP y reindexado opcional de Elasticsearch.

10.14 Neo4J

Neo4J¹⁵ modela usuarios, pedidos, productos, restaurantes, ubicaciones y repartidores como grafo. El punto de rutas usa una heurística de vecino más cercano para asignar pedidos a repartidores disponibles.

11 Configuración principal

Las variables se leen desde `.env` y desde el bloque común de `docker-compose.yml`.

Variable	Uso
DATABASE_ENGINE	Selecciona mongo o postgres.
POSTGRES_URL	URL SQLAlchemy de PostgreSQL.
MONGO_URL	URL de conexión al router mongos.
KEYCLOAK_URL	URL interna o externa de Keycloak.
KEYCLOAK_REALM	Realm de Keycloak: <code>restaurant-realm</code> .
KEYCLOAK_CLIENT_ID	Cliente OIDC usado por la API.
REDIS_URL	URL de Redis.
ELASTICSEARCH_URL	URL de Elasticsearch.
CACHE_TTL_SECONDS	TTL de caché general.
SEARCH_CACHE_TTL_SECONDS	TTL de caché de búsqueda.
SPARK_SCALE_FACTOR	Multiplicador de volumen para el job Spark.
AIRFLOW_PORT	Puerto de Airflow Webserver.
SUPERSET_PORT	Puerto de Superset.
ENABLE_DELIVERY_ROUTE_VALIDATION	Activa validación opcional de rutas en Airflow.

12 Modelo de datos transaccional

Esta sección describe la estructura de datos utilizada por el sistema para soportar las operaciones principales de la aplicación. El modelo transaccional representa las entidades necesarias para gestionar usuarios, restaurantes, menús, productos, mesas, reservaciones y pedidos, así como las relaciones existentes entre ellas. Su propósito es garantizar que la información operativa del negocio se almacene de forma organizada, consistente y consultable.

12.1 Entidades

Entidad	Campos principales
Role	role_id, role_name.
User	user_id, user_name, keycloak_id, role_id.
Restaurant	restaurant_id, restaurant_name, admin_id, restaurant_status.
Menu	menu_id, dish_name, category, description, price, restaurant_id.
Table	table_id, table_number, table_status, restaurant_id.
Order	order_id, table_id, client_id, order_type, restaurant_id.
OrderItem	order_item_id, order_id, menu_id, quantity, price.
Reservation	reservation_id, table_id, client_id, reservation_date, reservation_status.

12.2 Restricciones relevantes

- En menús no puede repetirse el mismo `dish_name` dentro del mismo restaurante.
- En mesas no puede repetirse el mismo `table_number` dentro del mismo restaurante.
- `keycloak_id` identifica al usuario en Keycloak y debe ser único si existe.
- `table_status` usa valores entre 0 y 2: ocupado, disponible o reservado, según la convención del código.
- Las operaciones de creación validan claves foráneas lógicas antes de persistir.

13 Capa DAO

La API usa `get_dao()` para seleccionar una implementación concreta:

- `PostgresDAO`: usa SQLAlchemy y sesiones transaccionales.
- `MongoDAO`: usa PyMongo, colecciones y contadores para generar IDs incrementales.

Esto permite que los routers no conozcan el motor activo. Por ejemplo, `dao.create_menu(...)` crea un documento en MongoDB o una fila en PostgreSQL según `DATABASE_ENGINE`.

Listing 1: Selección conceptual del DAO activo.

```
if DATABASE_ENGINE == "mongo":
    yield MongoDAO(get_mongo_db())
elif DATABASE_ENGINE == "postgres":
    yield PostgresDAO(db)
```

14 Autenticación y autorización

El sistema utiliza mecanismos de autenticación y autorización para validar la identidad de los usuarios y controlar el acceso a las funcionalidades según su rol. Esto permite proteger las rutas de la API, diferenciar permisos entre clientes y administradores, y asegurar que cada operación sea realizada únicamente por usuarios autorizados.

14.1 Flujo de registro

El registro crea primero el usuario en Keycloak con rol `client`. Luego crea el usuario local en la base activa. Si la creación local falla, se intenta revertir eliminando el usuario creado en Keycloak.

14.2 Flujo de login

El login envía usuario y contraseña al endpoint OIDC de Keycloak. Si las credenciales son válidas, Keycloak responde con un JSON que incluye `access_token`, `refresh_token`, expiración y metadatos del token.

14.3 Validación de token

Cada ruta protegida usa HTTPBearer. La API:

1. Extrae el token del header `Authorization`.
2. Consulta las llaves públicas JWKS de Keycloak.
3. Busca la llave con el `kid` del token.
4. Decodifica el JWT con algoritmo RS256.
5. Valida el `issuer`.
6. Devuelve el payload si todo es correcto.

14.4 Roles

- **client**: puede consultar recursos protegidos, crear pedidos y reservas.
- **admin**: puede crear, actualizar y eliminar restaurantes, menús y mesas; también puede administrar usuarios cuando la lógica del endpoint lo permite.

15 Convenciones generales de API

Esta sección define las reglas y criterios comunes que se utilizan en los endpoints del sistema. Estas convenciones permiten mantener una estructura uniforme en las solicitudes y respuestas de la API, facilitando su consumo, documentación, pruebas y mantenimiento. Entre los aspectos principales se incluyen las rutas base, los encabezados requeridos, los códigos de estado y el formato esperado para la comunicación entre clientes y servicios.

15.1 Base URLs

- API por Nginx: `http://localhost:8080/api`
- Search por Nginx: `http://localhost:8080/search`
- API directa: `http://localhost:8000`
- Documentación API directa: `http://localhost:8000/docs`
- Documentación por Nginx: `http://localhost:8080/api/docs`

15.2 Headers

Todas las rutas protegidas requieren:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

15.3 Códigos de estado

La siguiente tabla resume los códigos de estado HTTP utilizados por la API y su significado dentro del sistema.

Código	Interpretación
200	Operación exitosa con cuerpo de respuesta.
201	Recurso creado correctamente.
204	Operación exitosa sin cuerpo, normalmente eliminación.
400	Solicitud inválida o regla de negocio incumplida.
401	Token ausente, inválido, expirado o credenciales incorrectas.
403	Usuario autenticado sin permisos suficientes.
404	Recurso o dependencia no encontrada.
409	Conflicto de unicidad, por ejemplo plato o mesa duplicada.
422	Error de validación Pydantic: tipos, rangos o campos requeridos.
500	Error interno no controlado.
503	Servicio externo no disponible o mal configurado, especialmente Keycloak.

16 How-Tos

Los *How-Tos* son guías cortas orientadas a tareas. A diferencia de la referencia, aquí importa el flujo completo.

16.1 Registrar un cliente

1. Enviar `POST /auth/register` con `username`, `email` y `password`.
2. La API crea el usuario en Keycloak con rol `client`.
3. La API crea el usuario local en PostgreSQL o MongoDB.
4. La respuesta devuelve `user_id` y `username`.

16.2 Crear un plato de menú

1. Iniciar sesión como administrador usando `POST /auth/login`.
2. Enviar `POST /menus/` con el token admin.
3. La API valida rol, restaurante existente y unicidad de plato por restaurante.
4. La API guarda el menú, borra `menus:all` y borra cachés `search:products:*`.

16.3 Buscar productos

1. Iniciar sesión como cliente o admin.
2. Ejecutar GET `/products?q=pizza` en el servicio Search.
3. Redis se consulta primero.
4. Si no existe caché, Elasticsearch ejecuta `multi_match`.
5. La respuesta se guarda en Redis y se devuelve al cliente.

16.4 Reindexar Elasticsearch

1. Enviar POST `/reindex` con token válido.
2. El servicio Search recrea el índice `products`.
3. Consulta todos los menús desde el DAO activo.
4. Indexa cada producto y refresca el índice.
5. Elimina caché de búsqueda.

16.5 Ejecutar el pipeline analítico

1. Levantar Hive, Spark, Airflow y Superset con Docker Compose.
2. Abrir `http://localhost:8090`.
3. Activar y disparar el DAG `restaurant_olap_pipeline`.
4. Revisar Grid y logs de cada tarea.
5. Abrir Superset en `http://localhost:8088` para consultar dashboards.

17 API Reference

Esta sección usa el estilo de referencia de una documentación de plataforma: cada operación indica método, ruta, autenticación, parámetros, body, respuesta e interacción interna.

17.1 Auth

17.1.1 **POST** `/auth/register`

Descripción: registra un cliente nuevo.

Autenticación: no requiere token.

Request body

Campo	Tipo	Requerido	Descripción
username	string	Sí	3 a 64 caracteres. Nombre único local.
email	string	Sí	Email válido para Keycloak.
password	string	Sí	8 a 128 caracteres.

Response 201

```
{
  "message": "Usuario creado correctamente",
  "user_id": 2,
  "username": "cliente1"
}
```

Errores: 400 usuario duplicado; 503 Keycloak no configurado; 500 fallo creando usuario.

Interacción interna: Keycloak Admin API, DAO activo, tabla/colección **users**, tabla/colección **roles**.

17.1.2 **POST** /auth/login

Descripción: autentica credenciales contra Keycloak y devuelve tokens.

Autenticación: no requiere token.

Request body

Campo	Tipo	Requerido	Descripción
username	string	Sí	Usuario de Keycloak.
password	string	Sí	Contraseña del usuario.

Response 200: objeto OIDC de Keycloak con **access_token**, **refresh_token**, expiración, tipo de token y scope.

Errores: 401 credenciales inválidas; 403 acciones pendientes en Keycloak.

Interacción interna: endpoint OIDC **/realms/<realm>/protocol/openid-connect/token**.

17.2 Users

17.2.1 **GET** /users/me

Descripción: devuelve el usuario local asociado al token actual.

Autenticación: token válido.

Headers: Authorization: Bearer<access_token>.

Response 200

```
{
  "user_name": "admin",
  "role_id": 1,
  "user_id": 1
}
```

Errores: 401 token sin sub; 404 usuario no existe en base local.

Interacción interna: validación JWKS contra Keycloak y consulta **get_user_by_keycloak_id**.

17.2.2 **PUT** /users/{user_id}

Descripción: actualiza el perfil de un usuario.

Autenticación: dueño del recurso o admin.

Path params

Parámetro	Tipo	Descripción
user_id	integer	Identificador del usuario local.

Request body

```
{
  "user_name": "nuevo_nombre"
}
```

Response 200: UserResponse.

Errores: 400 username ocupado; 403 no es dueño/admin o intenta cambiar rol; 404 usuario inexistente; 503 no se pudo sincronizar Keycloak.

Interacción interna: Keycloak Admin API y DAO activo.

17.2.3 **DELETE** /users/{user_id}

Descripción: elimina cuenta local y usuario de Keycloak.

Autenticación: dueño del recurso o admin.

Response 204: sin body.

Errores: 401, 403, 404, 503.

Interacción interna: Keycloak Admin API y DAO activo.

17.3 Restaurants

17.3.1 **GET** /restaurants/

Descripción: lista restaurantes.

Autenticación: token válido.

Response 200: arreglo de RestaurantResponse.

Interacción interna: DAO activo.

17.3.2 **GET** /restaurants/{restaurant_id}

Descripción: obtiene un restaurante por ID.

Autenticación: token válido.

Response 200

```
{
  "restaurant_name": "Restaurante Central",
  "admin_id": 1,
  "restaurant_status": 1,
  "restaurant_id": 1
}
```


Errores: 404 restaurante no encontrado.

17.3.3 **POST** /restaurants/

Descripción: crea restaurante.

Autenticación: admin.

Request body

```
{  
  "restaurant_name": "Restaurante Central",  
  "admin_id": 1,  
  "restaurant_status": 1  
}
```

Response 201: RestaurantResponse.

Errores: 403 no admin; 404 administrador no existe; 422 body inválido.

Interacción interna: validación de rol, DAO activo, relación con users.

17.3.4 **PUT** /restaurants/{restaurant_id}

Descripción: actualiza parcialmente restaurante.

Autenticación: admin.

Request body: cualquiera de restaurant_name, admin_id, restaurant_status.

Errores: 400 body vacío; 403 no admin; 404 restaurante o admin no existe.

17.3.5 **DELETE** /restaurants/{restaurant_id}

Descripción: elimina restaurante.

Autenticación: admin.

Response 204: sin body.

Errores: 403 no admin; 404 restaurante no encontrado.

17.4 Menus

17.4.1 **GET** /menus/

Descripción: lista platos del menú.

Autenticación: token válido.

Response 200: arreglo de MenuResponse.

Cache: lee/escribe menus:all.

Interacción interna: Redis y DAO activo.

17.4.2 **GET** /menus/{menu_id}

Descripción: obtiene un plato por ID.

Autenticación: token válido.

Response 200

```
{
  "dish_name": "Pizza Margarita",
  "category": "Plato fuerte",
  "description": "Pizza con queso, tomate y albahaca",
  "price": 7200.0,
  "restaurant_id": 1,
  "menu_id": 1
}
```

Cache: lee/escribe menus:<menu_id>.

Errores: 404 plato no encontrado.

17.4.3 **POST** /menus/

Descripción: crea plato.

Autenticación: admin.

Request body

```
{
  "dish_name": "Pizza Margarita",
  "category": "Plato fuerte",
  "description": "Pizza con queso, tomate y albahaca",
  "price": 7200.0,
  "restaurant_id": 1
}
```

Response 201: MenuResponse.

Errores: 403 no admin; 404 restaurante no existe; 409 plato duplicado en restaurante.

Interacción interna: DAO activo, Redis, invalidación de menus:all y search:products:

*.

17.4.4 **PUT** /menus/{menu_id}

Descripción: actualiza parcialmente plato.

Autenticación: admin.

Request body: cualquiera de dish_name, category, description, price, restaurant_id.

Errores: 400 body vacío; 403; 404; 409.

Interacción interna: DAO activo e invalidación de caché de menú y búsqueda.

17.4.5 **DELETE** /menus/{menu_id}

Descripción: elimina plato.

Autenticación: admin.

Response 204: sin body.

Interacción interna: DAO activo e invalidación de menus:all, menus:<id> y search:products:.*.

17.5 Tables

17.5.1 GET /tables/

Descripción: lista mesas.

Autenticación: token válido.

Response 200: arreglo de TableResponse.

17.5.2 POST /tables/

Descripción: crea mesa.

Autenticación: admin.

Request body

```
{
  "table_number": 12,
  "table_status": 1,
  "restaurant_id": 1
}
```

Response 201: TableResponse.

Errores: 403 no admin; 404 restaurante no existe; 409 número de mesa duplicado en restaurante.

17.5.3 GET /tables/{table_id}

Descripción: obtiene mesa por ID.

Errores: 404 mesa no encontrada.

17.5.4 PUT /tables/{table_id}

Descripción: actualiza mesa parcialmente.

Autenticación: admin.

Errores: 400 body vacío; 403; 404; 409.

17.5.5 DELETE /tables/{table_id}

Descripción: elimina mesa.

Autenticación: admin.

Response 204: sin body.

17.6 Orders

17.6.1 POST /orders/

Descripción: crea pedido.

Autenticación: token válido.

Request body

```
{
  "table_id": 1,
  "client_id": 2,
  "order_type": "local",
  "restaurant_id": 1
}
```

Response 201: OrderResponse.

Errores: 404 mesa, cliente o restaurante no existe; 422 body inválido.

Interacción interna: DAO activo.

17.6.2 **GET** /orders/{order_id}

Descripción: obtiene pedido por ID.

Autenticación: token válido.

Errores: 404 pedido no encontrado.

17.7 Reservations

17.7.1 **POST** /reservations/

Descripción: crea reservación.

Autenticación: token válido.

Request body

```
{
  "table_id": 1,
  "client_id": 2,
  "reservation_date": "2026-06-25T19:30:00",
  "reservation_status": 1
}
```

Response 201: ReservationResponse.

Errores: 404 mesa o cliente no existe; 422 fecha o tipos inválidos.

17.7.2 **DELETE** /reservations/{reservation_id}

Descripción: cancela/elimina reservación.

Autenticación: token válido.

Response 204: sin body.

Errores: 404 reservación no encontrada.

17.8 Search

17.8.1 **GET** /products

Descripción: búsqueda textual de productos.

Autenticación: token válido.

Query params

Parámetro	Tipo	Requerido	Descripción
q	string	Sí	Texto a buscar en nombre, categoría o descripción.

Response 200: arreglo de productos indexados.

Cache: search:products:text:<q>.

Interacción interna: Redis y Elasticsearch.

17.8.2 GET /products/category/{category}

Descripción: busca productos por categoría exacta normalizada.

Autenticación: token válido.

Response 200: arreglo de productos indexados.

Cache: search:products:category:<category>.

17.8.3 POST /reindex

Descripción: reconstruye índice de productos.

Autenticación: token válido.

Response 200

```
{
  "message": "Productos reindexados correctamente",
  "total": 25
}
```

Interacción interna: DAO activo, Elasticsearch y Redis.

17.9 Graph

17.9.1 GET /graph/export

Descripción: exporta datos operacionales para Neo4J y Data Warehouse.

Autenticación: en la implementación actual no declara dependencia de token.

Response 200

```
{
  "roles": [],
  "users": [],
  "restaurants": [],
  "products": [],
  "orders": [],
  "order_items": []
}
```

Interacción interna: DAO activo. También calcula total de cada pedido usando order_items.

18 Contratos de API

18.1 Sistema

Endpoint	Auth	Respuesta
GET /ping	No	\{"message":"pong"\}.
GET /	No	Mensaje de API funcionando y versión.
GET /health	No	\{"status":"ok"\}.

18.2 Autenticación

18.2.1 POST /auth/register

Registra un usuario cliente en Keycloak y en la base activa.

Body

```
{
  "username": "cliente1",
  "email": "cliente1@example.com",
  "password": "password123"
}
```

Validaciones

- username: 3 a 64 caracteres.
- email: email válido.
- password: 8 a 128 caracteres.
- No debe existir otro usuario con el mismo username.
- Keycloak debe estar configurado y accesible.

Respuesta 201

```
{
  "message": "Usuario creado correctamente",
  "user_id": 2,
  "username": "cliente1"
}
```

Errores

- 400: campos faltantes o usuario duplicado.
- 503: Keycloak no configurado.
- 500: error creando usuario en Keycloak o en base de datos.

18.2.2 POST /auth/login

Obtiene tokens desde Keycloak.

Body

```
{
  "username": "admin",
  "password": "admin"
}
```

Respuesta 200

```
{
  "access_token": "<jwt>",
  "expires_in": 300,
  "refresh_expires_in": 1800,
  "refresh_token": "<jwt>",
  "token_type": "Bearer",
  "not-before-policy": 0,
  "session_state": "...",
  "scope": "profile email"
}
```

Interpretación: `access_token` se usa en rutas protegidas. `refresh_token` permite renovar sesión según la configuración de Keycloak.

Errores

- 401: credenciales inválidas o usuario inexistente.
- 403: acciones requeridas pendientes en Keycloak.

18.3 Usuarios

Endpoint	Entrada	Auth	Salida / reglas
GET /users/me	Ninguna	Token válido	Devuelve el usuario local asociado al <code>sub</code> del JWT.
PUT /users/id	UserUpdate	Dueño o admin	Actualiza <code>user_name</code> . No permite escalar rol. Sincroniza nombre con Keycloak.
DELETE /users/id	ID en path	Dueño o admin	Elimina usuario en Keycloak y base local. Responde 204.

UserUpdate

```
{
  "user_name": "nuevo_nombre",
  "role_id": 2
}
```

Aunque el esquema acepta `role_id`, el endpoint rechaza cambios de rol si difieren del rol actual.

18.4 Restaurantes

Endpoint	Entrada	Auth	Comportamiento
GET /restaurants/	Ninguna	Token válido	Lista todos los restaurantes.
GET /restaurants/id	ID	Token válido	Devuelve un restaurante o responde 404.
POST /restaurants/	RestaurantCreate	Admin	Crea restaurante si <code>admin_id</code> existe.
PUT /restaurants/id	RestaurantUpdate	Admin	Actualiza parcialmente; requiere al menos un campo.
DELETE /restaurants/id	ID	Admin	Elimina restaurante o responde 404.

RestaurantCreate

```
{
  "restaurant_name": "Restaurante Central",
  "admin_id": 1,
  "restaurant_status": 1
}
```

RestaurantResponse

```
{
  "restaurant_name": "Restaurante Central",
  "admin_id": 1,
  "restaurant_status": 1,
  "restaurant_id": 1
}
```

18.5 Menús

Endpoint	Entrada	Auth	Comportamiento
GET /menus/	Ninguna	Token válido	Lista menús. Usa Redis con llave <code>menus:all</code> .
GET /menus/id	ID	Token válido	Devuelve un plato. Usa Redis con llave <code>menus:<id></code> .
POST /menus/	MenuCreate	Admin	Crea plato si restaurante existe. Invalida caché.
PUT /menus/id	MenuUpdate	Admin	Actualiza parcialmente. Invalida caché de menú y búsqueda.

DELETE /menus/id	ID	Admin	Elimina plato e invalida caché.
---------------------	----	-------	---------------------------------

MenuCreate

```
{
  "dish_name": "Pizza Margarita",
  "category": "Plato fuerte",
  "description": "Pizza con queso, tomate y albahaca",
  "price": 7200.0,
  "restaurant_id": 1
}
```

Reglas

- price debe ser mayor que 0.
- restaurant_id debe existir.
- Si ya existe el mismo dish_name en el restaurante, responde 409.

18.6 Mesas

Endpoint	Entrada	Auth	Comportamiento
GET /tables/	Ninguna	Token válido	Lista mesas.
GET /tables/id	ID	Token válido	Devuelve mesa o 404.
POST /tables/	TableCreate	Admin	Crea mesa si restaurante existe.
PUT /tables/id	TableUpdate	Admin	Actualiza parcialmente.
DELETE /tables/id	ID	Admin	Elimina mesa.

TableCreate

```
{
  "table_number": 12,
  "table_status": 1,
  "restaurant_id": 1
}
```

Reglas

- table_number debe ser mayor que 0.
- table_status debe estar entre 0 y 2.
- No se permite repetir número de mesa dentro del mismo restaurante.

18.7 Pedidos

Endpoint	Entrada	Auth	Comportamiento
POST /orders/	OrderCreate	Token válido	Crea pedido validando mesa opcional, cliente y restaurante.
GET /orders/id	ID	Token válido	Devuelve pedido o 404.

OrderCreate

```
{
  "table_id": 1,
  "client_id": 2,
  "order_type": "local",
  "restaurant_id": 1
}
```

Interpretación

Un pedido representa la cabecera de la orden. Los ítems existen en el modelo y DAO, y son usados por exportaciones analíticas y de grafo, aunque no hay router CRUD público para `order_items` en la API actual.

18.8 Reservaciones

Endpoint	Entrada	Auth	Comportamiento
POST /reservations/	ReservationCreate	Token válido	Crea reserva si mesa y cliente existen.
DELETE /reservations/id	ID	Token válido	Cancela/elimina reserva. Responde 204.

ReservationCreate

```
{
  "table_id": 1,
  "client_id": 2,
  "reservation_date": "2026-06-25T19:30:00",
  "reservation_status": 1
}
```

Interpretación

`reservation_date` se envía como fecha/hora ISO 8601. `reservation_status` es entero no negativo y representa el estado de negocio definido por el sistema.

18.9 Búsqueda

El microservicio Search se publica normalmente bajo `/search`. Si se accede directo al contenedor, las rutas no incluyen prefijo.

Endpoint	Entrada	Auth	Comportamiento
GET /products?q=texto	Query q	Token válido	Busca productos en Elasticsearch por nombre, categoría o descripción. Usa Redis.
GET /products/category/{category}	Categoría	Token válido	Filtra por categoría exacta normalizada. Usa Redis.
POST /reindex	Ninguna	Token válido	Recrea índice products , indexa todos los menús y limpia caché de búsqueda.

Respuesta de búsqueda

```
[
  {
    "menu_id": 1,
    "dish_name": "Pizza Margarita",
    "category": "Plato fuerte",
    "description": "Pizza con queso, tomate y albahaca",
    "price": 7200.0,
    "restaurant_id": 1
  }
]
```

Respuesta de reindexado

```
{
  "message": "Productos reindexados correctamente",
  "total": 25
}
```

18.10 Exportación para grafo

18.10.1 GET /graph/export

Exporta datos operacionales para Neo4J y para el generador de seed del Data Warehouse.

Respuesta

```
{
  "roles": [],
  "users": [],
  "restaurants": [],
  "products": [],
  "orders": [],
  "order_items": []
}
```

Interpretación: cada arreglo contiene registros normalizados. En **orders**, la API calcula **total** sumando **price*quantity** de sus ítems.

19 Flujos de datos operacionales

19.1 Administrador agrega plato

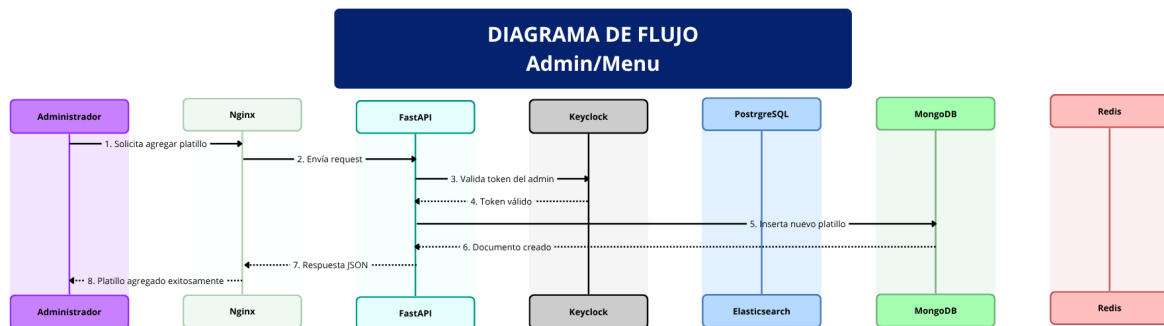


Figura 3: Flujo de datos para creación de menú por administrador.

El administrador envía una solicitud para agregar un platillo. Nginx redirige hacia FastAPI. La API valida el token con Keycloak y verifica rol **admin**. Después valida que el restaurante exista, crea el documento/fila en la base activa y limpia cachés afectadas. La respuesta JSON confirma el recurso creado.

19.2 Cliente busca productos

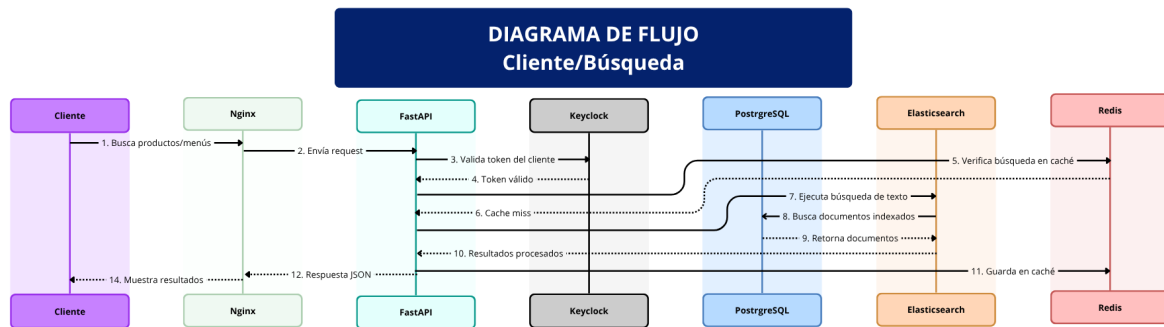


Figura 4: Flujo de datos para búsqueda de productos por cliente.

El cliente consulta productos. La API valida token, consulta Redis y, si hay *cache hit*, responde inmediatamente. Si hay *cache miss*, ejecuta búsqueda en Elasticsearch, procesa documentos encontrados, guarda el resultado en Redis y responde al cliente.

19.3 Cliente crea reservación

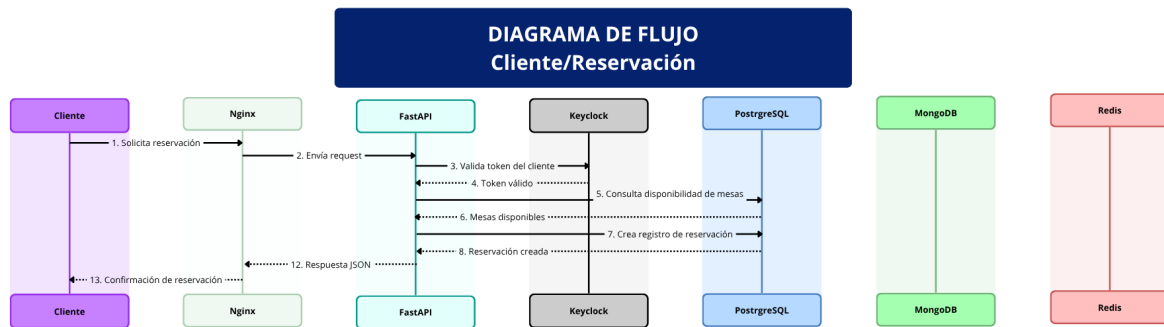


Figura 5: Flujo de datos para creación de reservación.

El cliente solicita una reserva. La API valida token, verifica que la mesa exista, verifica que el cliente exista y crea la reserva en la base activa. La respuesta confirma la reserva creada.

20 Ejecución local

20.1 Levantar el sistema completo

```
docker compose up -d --build
```

20.2 Verificar servicios

```
docker compose ps
docker compose logs -f keycloak
docker compose logs -f api
docker compose logs -f seed
```

20.3 Credenciales por defecto

Servicio	Usuario / password	URL
Keycloak Admin	admin/admin	http://localhost:8001/admin/master/console/
Usuario seed API	admin/admin	vía /auth/login
PostgreSQL	postgres/postgres123	localhost:5432
Superset	admin/admin	http://localhost:8088
Airflow	admin/admin	http://localhost:8090
Neo4J	neo4j/restaurant123	http://localhost:7474

21 Carpeta dataW: plataforma analítica completa

21.1 Objetivo de dataW

dataW implementa la parte analítica del proyecto. Su propósito es transformar datos operacionales en información agregada para análisis OLAP y dashboards.

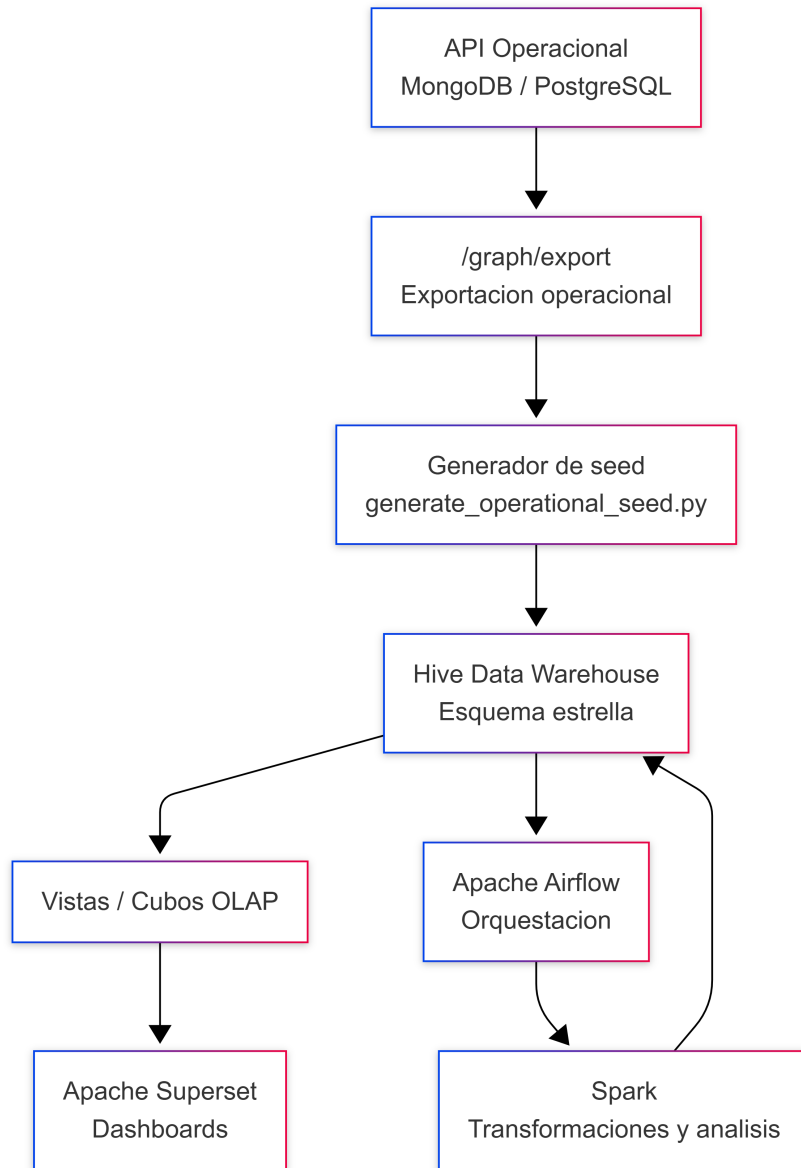


Figura 6: Flujo general de la plataforma analítica dataW.

Airflow coordina el flujo completo y Spark ejecuta análisis distribuidos complementarios.

21.2 Estructura de dataW

Ruta	Descripción
dataW/warehouse/	Esquema estrella, scripts Hive, seed operacional, ETL histórico y pruebas del Data Warehouse.
dataW/spark/	Job PySpark, salidas CSV/JSON y validación del procesamiento distribuido.

dataW/visualization/	Configuración de Superset, consultas SQL y catálogo de dashboards.
dataW/airflow/	DAG, scripts de orquestación, Dockerfile, estado del pipeline y pruebas de Airflow.

21.3 Data Warehouse en Hive

El Data Warehouse usa un esquema estrella. Las dimensiones describen contexto; los hechos almacenan métricas.

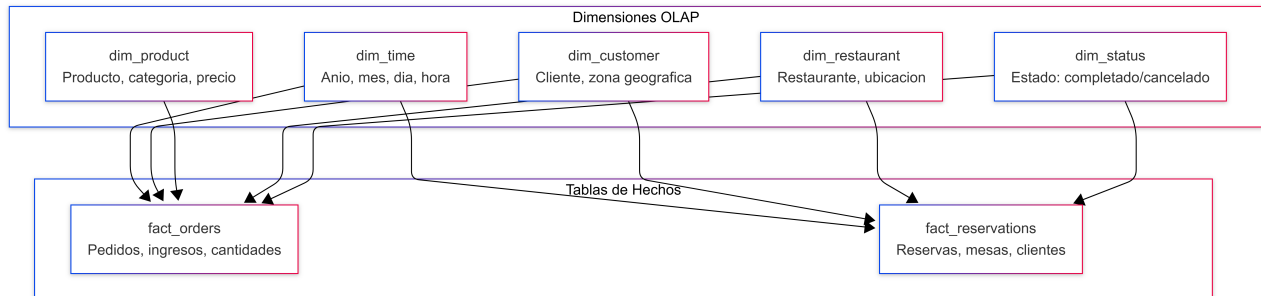


Figura 7: Modelo dimensional del Data Warehouse en Hive.

21.3.1 Dimensiones

Tabla	Propósito
dim_time	Fechas, día, semana, mes, trimestre, año, fin de semana y temporada.
dim_customer	Cliente, tipo, zona geográfica, nivel de lealtad, gasto total y actividad.
dim_product	Producto, categoría, precio, costo, margen y disponibilidad.
dim_restaurant	Restaurante, ubicación, zona, capacidad, contacto y estado.
dim_status	Estados de órdenes y reservaciones.

21.3.2 Hechos

Tabla	Métricas
fact_orders	Cantidad, precio unitario, total, descuento, neto, impuesto, final, hora de orden y hora de entrega.
fact_reservations	Tamaño del grupo, duración, ocupación de mesa, no-show, fecha de reserva, check-in y check-out.

21.3.3 Vistas OLAP actuales

El archivo dataW/warehouse/schemas/hive_dashboard_views.sql crea las vistas usadas por los dashboards de Superset:

- cubo_ingresos_mes_categoria
- cubo_actividad_clientes_zona

■ cubo_ordenes_completadas_canceladas

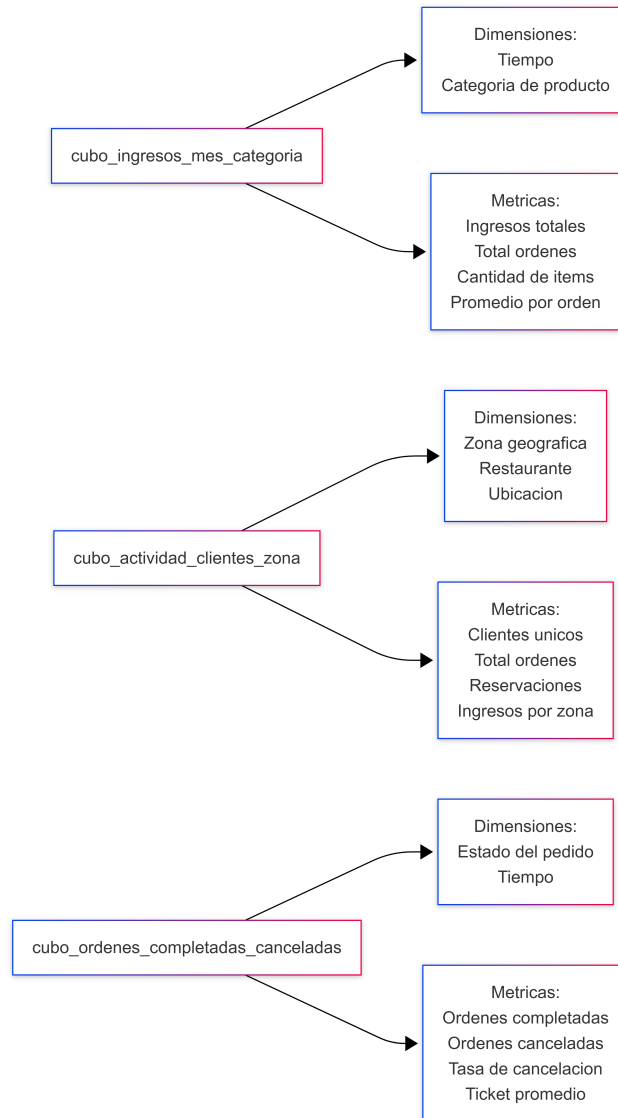


Figura 8: Cubos OLAP utilizados para análisis y dashboards.

Ejemplo de consulta OLAP

```

SELECT
    year,
    month_name,
    category,
    ingresos_totales
FROM cubo_ingresos_mes_categoria
ORDER BY year, month, category;

```

21.4 Generación de seed operacional

El servicio `warehouse-seed-generator` ejecuta:

```
dataW/warehouse/scripts/generate_operational_seed.py
```

Este script consulta:

```
http://api/graph/export
```

y genera:

```
dataW/warehouse/generated/operational_seed.hql
```

El seed transforma usuarios, restaurantes, productos, pedidos e ítems en filas dimensionales y de hechos para Hive. Si faltan `order_items`, el script puede completar datos derivados por restaurante para mantener un seed analítico utilizable.

21.5 Spark

El job `dataW/spark/jobs/analytics_job.py` demuestra procesamiento distribuido con tres análisis:

1. Tendencias de consumo por mes, categoría y producto.
2. Horarios pico por día, hora y categoría.
3. Crecimiento mensual con funciones ventana.

21.5.1 Entradas simuladas y escalamiento

El job crea DataFrames base para tiempo, productos, restaurantes, órdenes y reservaciones. Luego escala hechos con:

```
SPARK_SCALE_FACTOR=1000
```

Con el valor por defecto se generan 6000 órdenes y 5000 reservaciones.

21.5.2 Salidas

Salida	Contenido
<code>output/consumption_trends</code>	CSV de tendencias de consumo.
<code>output/peak_hours</code>	CSV de horarios pico.
<code>output/monthly_growth</code>	CSV de crecimiento mensual.
<code>output/validation_summary.json</code>	Conteos de entrada y salida; evidencia de ejecución.

21.6 Superset

Superset construye dashboards consultando directamente Hive. La herramienta no consume la API transaccional; consume vistas OLAP ya materializadas lógicamente.

21.6.1 Dashboards requeridos

Dashboard	Vista	Indicadores
Ingresos por mes y categoría	cubo_ingresos_mes_categoria	Ingresos totales, órdenes, ingreso por categoría, serie mensual.
Actividad de clientes por zona	cubo_actividad_clientes_zona	Clientes únicos, órdenes, ingresos por zona, ubicación.
Pedidos completados vs cancelados	cubo_ordenes_completadas_canceladas	Cantidad por estado, tasa de cancelación, monto asociado.

21.7 Airflow

Airflow ejecuta el DAG `restaurant_olap_pipeline`. Su schedule es diario:

```
@daily
```

21.7.1 Tareas del DAG

```
start
-> extract_from_source
-> run_spark_transformations
-> load_to_data_warehouse
-> validate_warehouse
-> validate_delivery_routes_optional
-> check_product_catalog_changes
-> reindex_elasticsearch_if_needed / skip_reindex
-> finish
```

21.7.2 Descripción de tareas

Tarea	Función
<code>extract_from_source</code>	Verifica conectividad a PostgreSQL y MongoDB. Puede continuar si <code>ALLOW_SOURCE_UNAVAILABLE=true</code> .
<code>run_spark_transformations</code>	Ejecuta el job PySpark en un contenedor conectado al cluster Spark.
<code>load_to_data_warehouse</code>	Crea base, esquema estrella, carga seed si corresponde y crea vistas OLAP.
<code>validate_warehouse</code>	Verifica existencia y conteos positivos en hechos y cubos requeridos.
<code>validate_delivery_routes_optional</code>	Ejecuta validación local de rutas si <code>ENABLE_DELIVERY_ROUTE_VALIDATION=true</code> .
<code>check_product_catalog_changes</code>	Calcula hash de <code>dim_product</code> para saber si cambió el catálogo.

<code>reindex_elasticsearch_if_needed</code>	Llama POST <code>http://search/reindex</code> si el catálogo cambió o si se fuerza.
<code>skip_reindex</code>	Rama usada cuando no hay cambios.

22 Neo4J y asignación de rutas

22.1 Modelo de grafo

El grafo contiene:

- Usuario
- Producto
- Pedido
- Restaurante
- Ubicacion
- Repartidor

Relaciones principales:

- `(:Usuario)-[:REALIZO]->(:Pedido)`
- `(:Pedido)-[:CONTIENE]->(:Producto)`
- `(:Pedido)-[:SALE_DE]->(:Restaurante)`
- `(:Pedido)-[:ENTREGAR_EN]->(:Ubicacion)`
- `(:Repartidor)-[:ASIGNADO_A]->(:Pedido)`

22.2 Heurística de vecino más cercano

El algoritmo local:

1. Carga repartidores, pedidos y ubicaciones desde JSON.
2. Calcula distancia entre ubicaciones con Haversine.
3. Procesa repartidores por rondas.
4. Para cada repartidor elige el pedido pendiente más cercano.
5. Actualiza la ubicación actual del repartidor.
6. Continúa hasta agotar pedidos o capacidad.

23 Pruebas y validación

23.1 Pruebas unitarias

```
pytest backend/tests/unit
```

23.2 Pruebas de integración

```
docker compose up -d --build
pytest backend/tests/integration
```

23.3 Validación Spark

```
.\dataW\spark\tests\run_full_spark_validation.ps1
```

23.4 Validación Data Warehouse

```
.\dataW\warehouse\tests\run_full_dw_validation.ps1
```

23.5 Validación visualización

```
.\dataW\visualization\tests\run_full_visualization_validation.ps1
```

23.6 Validación Airflow

```
python dataW\airflow\test\test_airflow_point4_requirements.py
```

23.7 Validación Neo4J

```
python .\neo4j\delivery_assignment.py  
python .\neo4j\test_delivery_routes.py
```

24 Seguridad

- Las rutas protegidas requieren JWT válido.
- Las operaciones administrativas validan rol **admin**.
- La API no confía solo en datos del body; revisa existencia de dependencias en la base.
- Keycloak centraliza credenciales y emisión de tokens.
- Los tokens se validan criptográficamente contra JWKS.
- Nginx puede actuar como punto de control para exponer solo rutas necesarias.

25 Consideraciones de consistencia

- El registro de usuario realiza dos escrituras: Keycloak y base local. Si falla la segunda, intenta compensar eliminando en Keycloak.
- El caché se invalida cuando se modifican menús porque afecta tanto listados como búsquedas.
- Elasticsearch no es la fuente de verdad; se reconstruye desde menús mediante **/reindex**.
- Hive tampoco es fuente transaccional; representa una copia analítica.
- Airflow evita duplicar cargas revisando si **fact_orders** ya tiene datos.

26 Limitaciones conocidas

- No existe router público completo para `order_items`; el modelo y DAO sí existen.
- Las reservas no tienen endpoint `GET` ni `PUT` público en la implementación actual.
- Algunos cubos mencionados en guías históricas existen como diseño conceptual, pero el archivo activo de dashboards crea tres vistas OLAP requeridas.
- El proyecto de rutas usa ubicaciones proyectadas o simuladas porque la base operacional no almacena coordenadas reales de clientes/restaurantes.
- La validación de audiencia del JWT está desactivada porque Keycloak no incluye audiencia por defecto sin protocol mapper.

27 Conclusión

El proyecto integra un sistema transaccional robusto con una plataforma analítica completa. FastAPI centraliza reglas de negocio y contratos REST; Keycloak protege el acceso; PostgreSQL y MongoDB permiten persistencia relacional o documental; Redis y Elasticsearch optimizan lectura y búsqueda; Nginx simplifica exposición; y `dataW` convierte los datos operacionales en análisis mediante Hive, Spark, Superset y Airflow. La arquitectura permite demostrar bases distribuidas, APIs, caché, búsqueda, OLAP, orquestación y grafos dentro de un mismo ecosistema reproducible.

28 Apéndice A: resumen de endpoints

Método	Ruta	Entrada	Respuesta exitosa
GET	/ping	—	\{"message":"pong"\}
GET	/health	—	\{"status":"ok"\}
POST	/auth/register	RegisterRequest	RegisterResponse, 201
POST	/auth/login	LoginRequest	Tokens Keycloak
GET	/users/me	Token	UserResponse
PUT	/users/{id}	UserUpdate	UserResponse
DELETE	/users/{id}	ID	204
GET	/restaurants/	Token	Lista RestaurantResponse
GET	/restaurants/{id}	ID	RestaurantResponse
POST	/restaurants/	RestaurantCreate	RestaurantResponse, 201
PUT	/restaurants/{id}	RestaurantUpdate	RestaurantResponse
DELETE	/restaurants/{id}	ID	204
GET	/menus/	Token	Lista MenuResponse
GET	/menus/{id}	ID	MenuResponse
POST	/menus/	MenuCreate	MenuResponse, 201
PUT	/menus/{id}	MenuUpdate	MenuResponse
DELETE	/menus/{id}	ID	204
GET	/tables/	Token	Lista TableResponse
GET	/tables/{id}	ID	TableResponse

POST	/tables/	TableCreate	TableResponse, 201
PUT	/tables/{id}	TableUpdate	TableResponse
DELETE	/tables/{id}	ID	204
POST	/orders/	OrderCreate	OrderResponse, 201
GET	/orders/{id}	ID	OrderResponse
POST	/reservations/	ReservationCreate	ReservationResponse, 201
DELETE	/reservations/{id}	ID	204
GET	/products?q=	Query q	Lista de productos indexados
GET	/products/category/{category}	Categoría	Lista de productos indexados
POST	/reindex	Token	Mensaje y total indexado
GET	/graph/export	—	Exportación operacional para grafo/DW

29 Apéndice B: archivos fuente relevantes

Archivo	Uso
backend/app/api_main.py	Aplicación FastAPI transaccional.
backend/app/search_main.py	Aplicación FastAPI de búsqueda.
backend/database.py	Selección de motor y dependencias de base.
backend/dao.py	DAO común para PostgreSQL y MongoDB.
backend/app/routers/*.py	Rutas REST.
backend/schemas/*.py	Contratos Pydantic.
backend/models/*.py	Modelos SQLAlchemy.
dbs/mongo/*.js	Inicialización, índices y sharding de MongoDB.
dbs/postgres/init.sql	Roles iniciales de PostgreSQL.
dataW/warehouse/schemas/schema_star.sql	Esquema estrella Hive.
dataW/warehouse/schemas/hive_dashboard_views.sql	Vistas OLAP activas.
dataW/spark/jobs/analytics_job.py	Procesamiento Spark.
dataW/airflow/dags/restaurant_olap_pipeline.py	DAG principal de Airflow.
dataW/visualization/dashboards/dashboard_catalog.md	Diseño de dashboards.
neo4j/delivery_assignment.py	Heurística local de rutas.

30 Referencias tecnológicas

- FastAPI: <https://fastapi.tiangolo.com/>
- Keycloak: <https://www.keycloak.org/documentation>
- PostgreSQL: <https://www.postgresql.org/docs/>
- MongoDB: <https://www.mongodb.com/docs/>
- Redis: <https://redis.io/docs/latest/>
- Elasticsearch: <https://www.elastic.co/guide/>

- Docker: <https://docs.docker.com/>
- Kubernetes: <https://kubernetes.io/docs/>
- Apache Hive: <https://hive.apache.org/>
- Apache Spark: <https://spark.apache.org/docs/latest/>
- Apache Superset: <https://superset.apache.org/docs/intro>
- Apache Airflow: <https://airflow.apache.org/docs/>
- Neo4J: <https://neo4j.com/docs/>

31 Glosario de acrónimos

Acrónimo	Significado	Uso en el proyecto
API	Application Programming Interface	Interfaz que permite la comunicación entre clientes externos y el sistema mediante endpoints HTTP.
REST	Representational State Transfer	Estilo de arquitectura usado por la API para exponer recursos mediante métodos HTTP como GET, POST, PUT y DELETE.
HTTP	Hypertext Transfer Protocol	Protocolo utilizado para enviar solicitudes y respuestas entre clientes, Nginx y los servicios FastAPI.
JWT	JSON Web Token	Token firmado emitido por Keycloak para autenticar usuarios y autorizar el acceso a rutas protegidas.
DAO	Data Access Object	Capa de acceso a datos que separa la lógica de negocio de la persistencia en PostgreSQL o MongoDB.
DTO	Data Transfer Object	Objeto utilizado para transportar datos entre capas del sistema o entre cliente y servidor. En el proyecto se relaciona con esquemas de entrada y salida.
CRUD	Create, Read, Update, Delete	Conjunto de operaciones básicas sobre recursos como usuarios, restaurantes, menús, mesas, pedidos y reservaciones.
DB	Database	Base de datos utilizada para almacenar información transaccional o analítica.
SQL	Structured Query Language	Lenguaje utilizado para consultar y manipular datos en bases relacionales y sistemas compatibles como PostgreSQL y Hive.
NoSQL	Not Only SQL	Modelo de bases de datos no relacionales, usado en el proyecto mediante MongoDB.
OLAP	Online Analytical Processing	Procesamiento orientado al análisis de datos agregados dentro del Data Warehouse.

OLTP	Online Transaction Processing	Procesamiento transaccional usado por la API para operaciones del sistema como pedidos, reservas y gestión de restaurantes.
DW	Data Warehouse	Almacén de datos analítico utilizado para consolidar información y generar consultas OLAP.
ETL	Extract, Transform, Load	Proceso de extracción, transformación y carga de datos hacia el Data Warehouse.
DAG	Directed Acyclic Graph	Estructura usada por Airflow para definir el orden de ejecución de tareas del pipeline analítico.
BI	Business Intelligence	Conjunto de prácticas y herramientas para analizar datos del negocio mediante dashboards e indicadores.
UI	User Interface	Interfaz visual utilizada para interactuar con servicios como Superset, Airflow, Keycloak o Neo4J Browser.
URL	Uniform Resource Locator	Dirección utilizada para acceder a endpoints, servicios web o documentación de tecnologías.
URI	Uniform Resource Identifier	Identificador de un recurso; se usa en rutas, endpoints y conexiones entre servicios.
JSON	JavaScript Object Notation	Formato de intercambio de datos usado en requests, responses, tokens y archivos de configuración.
YAML	YAML Ain't Markup Language	Formato utilizado en manifiestos de Kubernetes y archivos de configuración.
HQL	Hive Query Language	Lenguaje similar a SQL utilizado para crear y consultar tablas en Hive.
JDBC	Java Database Connectivity	Tecnología usada para conectar herramientas externas con bases de datos, como Superset con HiveServer2.
ODBC	Open Database Connectivity	Interfaz estándar para conectar aplicaciones con bases de datos.
JVM	Java Virtual Machine	Entorno de ejecución utilizado por herramientas del ecosistema Java como Spark, Hive y algunos conectores.
JWKS	JSON Web Key Set	Conjunto de llaves públicas usado para validar la firma de tokens JWT emitidos por Keycloak.
OIDC	OpenID Connect	Protocolo de autenticación usado por Keycloak para emitir tokens y manejar sesiones de usuario.
RBAC	Role-Based Access Control	Modelo de control de acceso basado en roles, usado para diferenciar permisos entre usuarios admin y client.

TTL	Time To Live	Tiempo durante el cual un dato permanece válido en caché, como las respuestas guardadas en Redis.
CSV	Comma-Separated Values	Formato tabular usado para salidas de procesamiento o intercambio de datos.
PDF	Portable Document Format	Formato usado para documentos de referencia, enunciados y entregables.
CLI	Command Line Interface	Interfaz de línea de comandos usada para ejecutar Docker, pruebas, scripts y herramientas del proyecto.
SSH	Secure Shell	Protocolo usado para acceso remoto seguro a servidores, si el despliegue se realiza en infraestructura externa.
DNS	Domain Name System	Sistema que traduce nombres de dominio a direcciones IP; puede intervenir en despliegues con servicios expuestos.
IP	Internet Protocol	Protocolo de red usado para identificar dispositivos y servicios dentro o fuera de la red Docker/Kubernetes.
TCP	Transmission Control Protocol	Protocolo de transporte usado por servicios como PostgreSQL, MongoDB, Redis, Elasticsearch y HiveServer2.
CRUD API	Create, Read, Update, Delete API	API que permite crear, consultar, actualizar y eliminar recursos del sistema.

32 Referencias tecnológicas

1. **FastAPI**. Documentación oficial. Disponible en: <https://fastapi.tiangolo.com/>
2. **Keycloak**. Documentación oficial. Disponible en: <https://www.keycloak.org/documentati on>
3. **PostgreSQL**. Documentación oficial. Disponible en: <https://www.postgresql.org/docs/>
4. **MongoDB**. Documentación oficial. Disponible en: <https://www.mongodb.com/docs/>
5. **Redis**. Documentación oficial. Disponible en: <https://redis.io/docs/latest/>
6. **Elasticsearch**. Documentación oficial. Disponible en: <https://www.elastic.co/guide/>
7. **Nginx**. Documentación oficial. Disponible en: <https://nginx.org/en/docs/>
8. **Docker**. Documentación oficial. Disponible en: <https://docs.docker.com/>
9. **Docker Compose**. Documentación oficial. Disponible en: <https://docs.docker.com/compose/>
10. **Kubernetes**. Documentación oficial. Disponible en: <https://kubernetes.io/docs/>
11. **Apache Hive**. Sitio oficial. Disponible en: <https://hive.apache.org/>
12. **Apache Spark**. Documentación oficial. Disponible en: <https://spark.apache.org/docs/latest/>
13. **Apache Superset**. Documentación oficial. Disponible en: <https://superset.apache.org/docs/intro>
14. **Apache Airflow**. Documentación oficial. Disponible en: <https://airflow.apache.org/docs/>
15. **Neo4J**. Documentación oficial. Disponible en: <https://neo4j.com/docs/>